

JAVA PROJECT EVALUATION

Telecom Customer & Subscription Management System (TCSMS)

Study notes for explaining multithreading, functional interfaces, lambda expressions, Stream API, and OOP with evidence from the project source.

Quick introduction

This Java console application manages telecom customers, plans, subscriptions, usage, billing, payments, complaints, and reports. The code separates console workflows, business services, database access (DAO), domain models, and background schedulers. The examples below are taken from the current files in **src/main/java**.

One-minute evaluation answer

“My project uses Java concurrency for background work such as processing usage batches, scheduling billing, scanning overdue bills, and handling notifications. Lambdas provide compact implementations of Runnable, Callable, Predicate, and Comparator. Functional interfaces define the expected behavior, and lambdas supply that behavior. The Stream API processes in-memory lists for filtering, sorting, grouping, and totals. OOP appears through encapsulated model fields, service and DAO abstractions, payment strategy implementations, and specialized exceptions.”

Keep the distinction clear: a stream pipeline is not automatically multithreaded. The shown calls use `stream()`, which is sequential; concurrency is provided separately by `ExecutorService` and `ScheduledExecutorService`.

1. Multithreading and concurrency

Multithreading lets a program make progress on more than one task at a time. Java executors manage worker threads so the program can submit tasks without manually creating a `Thread` for every job. It is useful for independent or time-consuming work, especially background database and notification tasks.

Where it appears in this project

Project location	What happens and why
UsageProcessor.java: 30-32, 39-63	A fixed pool of four workers accepts one Runnable per usage batch (49-62). The shared processed count is protected by synchronized (54-56, 89-92). This lets independent batches write concurrently while protecting the counter.
PaymentNotificationService.java: 24-32, 38-58, 64-76	Three workers poll a bounded BlockingQueue and process notifications. Payment flow can enqueue work quickly; workers perform it in the background. A full queue is handled explicitly (71-76).
BillingScheduler.java: 26-32, 41-75	A ScheduledExecutorService runs a billing task repeatedly with <code>scheduleAtFixedRate</code> (74). This automates billing without requiring an administrator to start each run manually.
AccountMonitor.java: 30-37, 44-80, 91-102	A two-thread pool runs Callable tasks and retrieves results through <code>Future.get</code> with a timeout. Callable returns a value; Future represents the pending result.
DBConnection.java: 33	The synchronized singleton accessor serializes instance creation. This protects creation of the shared connection manager, not every database operation.

2. Functional interfaces

A functional interface has exactly one abstract method (SAM: single abstract method). It is a target type for a lambda. It may also contain default or static methods. Runnable and Callable are concurrency interfaces; Predicate, Comparator, and Supplier express reusable behavior.

Project examples

Type	Abstract method idea	Project evidence and purpose
Runnable	void run()	UsageProcessor.java:49 and PaymentNotificationService.java:41. Runs work without returning a result.
Callable<T>	T call()	AccountMonitor.java:48, 57, 111. Tasks return overdue bills or a count; the executor provides a Future.
Predicate	boolean test(T)	PlanServiceImpl.java:36, 45, 54. Encapsulates conditions such as plan name contains a keyword or price is within budget.
Comparator	int compare(T,T)	PlanServiceImpl.java:63 and ReportServiceImpl.java:45-48. Defines ordering for prices or customer totals.
Supplier	T get()	PlanServiceImpl.java:78 uses the Supplier overload of orElseThrow; the exception is created only when no plan is found.

Important interview distinction

PaymentStrategy.java:8-10 is an ordinary interface, not a functional interface: it declares two abstract methods. It is used for polymorphism and the Strategy pattern. CustomerService.java is also a regular service contract. Do not claim these can be implemented by one lambda.

3. Lambda expressions

A lambda is a short expression or block of code that provides the implementation for a functional interface. The arrow separates parameters from the behavior: parameters -> body. Its type is usually inferred from the method argument or variable declaration.

Examples from the code

```
// Runnable: no result
executor.submit(() -> usageDAO.saveBatch(batch));

// Predicate: true when this plan matches
Predicate<TelecomPlan> withinBudget = p -> p.getMonthlyRental() <= maxPrice;

// Callable: computes and returns a value
Callable<Integer> task = () -> countOverdueBills();

// Comparator logic inside ReportServiceImpl
(c1, c2) -> Double.compare(totalFor(c2), totalFor(c1))
```

Actual examples: UsageProcessor.java:49-62; PlanServiceImpl.java:36, 45, 54; AccountMonitor.java:48-53; ReportServiceImpl.java:45-48. Lambdas reduce anonymous-class boilerplate and make the rule passed to an API visible at the call site.

4. Stream API

A Stream is a pipeline for processing items from a source such as a List. Intermediate operations such as filter, map, and sorted describe transformations; a terminal operation such as collect, sum, or forEach runs the pipeline and produces an outcome. Streams do not store the items and do not inherently create threads.

Pipeline example from plan search

```
plans.stream()           // get a stream from the list
    .filter(withinBudget) // keep affordable plans
    .collect(Collectors.toList()); // make a result list
```

Project examples and their business meaning

Location	Operation	Meaning
PlanServiceImpl.java:36-39	filter + collect	Return plans whose name matches the search.
PlanServiceImpl.java:63-71	sorted + collect	Sort plans by monthly rental, ascending or descending.
SubscriptionServiceImpl.java:48-49	anyMatch	Short-circuit check for an existing active subscription to the same plan.
ReportServiceImpl.java:34-50	groupingBy + summingDouble; sorted + limit	Sum bill values per customer, order customers by total, return top ten.
ReportServiceImpl.java:63-64	groupingBy	Build a map from city to customers.
ReportServiceImpl.java:76-81	filter + groupingBy + summarizingDouble	Summarize paid bill amounts by billing month (count, sum, min, max, average).
ReportServiceImpl.java:95-98	filter + mapToDouble + sum	Add paid bill amounts to calculate total revenue.

5. OOP concepts used

Object-oriented programming models a system as objects that combine state (data) and behavior (methods). These are clear examples to explain in this project:

Concept	Meaning in plain language	Project evidence
Encapsulation	Keep an object's state inside it and expose access through methods.	Customer.java:6-20 declares fields private; getters/setters begin at 44. A Customer object groups customer data and operations such as getFullName (56).
Abstraction	Expose a contract while hiding implementation details.	CustomerService.java:9-15 defines operations; CustomerServiceImpl.java:18 implements business rules. CustomerDAO is a persistence contract implemented by CustomerDAOImpl.
Inheritance	A specialized class receives behavior/type from a parent class.	ValidationException.java:3 extends TelecomException.java:3; the specialized validation error is still a TelecomException.
Polymorphism	Code can use a shared parent/interface while a concrete type supplies behavior.	PaymentStrategy.java defines common payment operations; UpiPaymentStrategy.java:5-20 implements them. Card and net-banking strategies provide alternate implementations. Factory selects one in PaymentStrategyFactory.java:13-26.
Composition / dependency	An object uses other objects to do its work.	CustomerServiceImpl.java:20-23 holds a CustomerDAO dependency and delegates persistence to it. This separates business rules from SQL.

6. Short viva questions

Q: Is a lambda the same thing as a functional interface?

A: No. The interface defines one abstract method; the lambda is one concise implementation of that method.

Q: Does stream() run in parallel?

A: No. stream() is sequential. parallelStream() or another concurrency mechanism is needed for parallel work; this project uses executors for background tasks.

Q: Why use Callable instead of Runnable?

A: Callable returns a result and may throw checked exceptions. Runnable has a void run() method and does not return a task result.

Q: Where is synchronization needed?

A: UsageProcessor protects updates and reads of the shared totalProcessed counter with the same lock. Otherwise multiple worker threads could overwrite each other's increments.

Q: Why use payment strategies?

A: Each payment channel has its own validation and processing behavior. The caller can program to PaymentStrategy and choose a concrete channel without mixing every channel's rules into one large conditional.

Revision note

Line references were checked against the Java files in this workspace when these notes were generated. Line numbers may shift if the source is edited. The existing reports/Java_Modern_Features_Evaluation.md was not used as authority for claims; examples were verified in source.